Prefix-Cache Radix Tree

Skip the prompt work an LLM server has already done: match, pin, insert, evict

Tracks: Trees (trie, radix tree) and caching (LRU, min-heap, reference counting) **Reading time:** about 30 minutes, longer if you try it yourself first **Code:** `src/PrefixCache/RadixPrefixCache.cs`. **Tests:** 26, run with `dotnet test`

---

0. The problem

> "Our LLM server handles thousands of requests a minute. Most prompts start the same way: the same system prompt, the same tool definitions, the same conversation so far. Without prefix caching, the model has to run prefill over every prompt token before it writes its first token, and on long prompts that can be a big part of the wait. Write the component that remembers which prompt prefixes already have their KV cache computed, so a new request only pays for the part that is new. GPU memory is limited, so it has to evict. And it must never evict KV that a running request is still using."

Try it before you read on. Hints, if you need them:

▼ Hint 1
"How much of this prompt have I seen before?" is a longest-prefix question. Which data structure answers that by walking down one symbol at a time?

▼ Hint 2
A system prompt is 3,000 tokens with no branching inside it. Does it really need 3,000 nodes?

▼ Hint 3
If you evict an inner node, what happens to everything below it? And what if two running requests share the same path?

---

1. Why this problem exists

Why reuse is correct at all. In a decoder-only model, attention is causal: token i only looks at tokens $0..i$. So the keys and values computed for token i, at every layer, depend only on the tokens before it. Two prompts that share their first n tokens have identical KV for those n tokens. Compute it once, reuse it.

Who does this in production:

System	What it does	The structure	Source
SGLang (RadixAttention)	Keeps the KV cache of past prompts in a radix tree over token ids. A new request reuses its longest cached prefix and computes only the rest	Radix tree, LRU over leaves, a reference count per node. This repo implements this shape	SGLang paper, NeurIPS 2024; `radix_cache.py`

System	What it does	The structure	Source
vLLM (automatic prefix caching)	Same goal. Each full KV block is hashed together with everything before it, and looked up in a hash table	Hash table of blocks, LRU over blocks with a reference count of 0	vLLM design doc
NVIDIA Dynamo	The router keeps a radix tree of which worker holds which prefixes, and sends a request where most of its prompt is already cached	Radix tree, one level above the servers	Dynamo router design
llm-d	Scores each pod by the longest run of the request's blocks it already has cached	Hash-keyed index with LRU	llm-d KV indexer
Anthropic, OpenAI, Google Gemini, DeepSeek	Prompt caching in their APIs: a repeated prompt beginning is billed at a fraction of the normal input price	Prefix matching, server side	Anthropic; OpenAI; Gemini; DeepSeek

The same building blocks outside AI:

- Linux looks up IPv4 routes in a trie, as a longest-prefix match (`fib_trie`).
- Redis keeps Streams in a radix tree (rax) and evicts with an approximate LRU (docs).
- PostgreSQL pins buffer pages with a reference count and only reuses a page whose count is 0 (buffer manager).

Why it matters more now: agents make many model calls per task, and every call starts with the same instructions and tools. Prefill on those shared beginnings is where a lot of GPU time goes, and it's also why prompt order became something API users have to think about.

---

2. Questions that shape the design (and the answers assumed here)

Ask	Why it changes the design	Assume
Match per token or per block?	A tree matches any length. Blocks fit a paged KV allocator but only reuse whole blocks	Per token (SGLang's choice). Blocks are compared in section 6.4
What is "capacity"?	GPU memory is spent per token of KV	Capacity in **tokens**
Who can be evicted?	Inner nodes are shared beginnings that their children depend on	**Leaves only**, least recently used first
What about requests still running?	Their KV must not disappear mid-request	A **reference count** per node; only 0 can be evicted
Two requests on the same path at once?	A yes/no flag would unpin too early	A **count**, not a flag
The tail doesn't fit even after evicting?	Refuse, or cache part of it	Cache **as much as fits**: a shorter prefix is still useful
Thread safety?	Locks on every node, or one owner	**One owner** (the scheduler loop), no locks
Must the order be deterministic?	Tests and replays	A logical clock for LRU, not wall time

---

3. The mental model

How one request goes through the cache:

1. `Acquire` walks down from the root, matching the request's tokens. If the match ends inside an edge, it splits the edge there. Every node on the matched path gets pinned (reference count plus 1).
2. The existing backend runs prefill only for the tokens after the match.
3. `Insert` adds the uncached tail as one new leaf. If there isn't room, it first evicts unpinned leaves, oldest first.
4. The model generates the answer.
5. `Dispose` unpins the path (reference count minus 1). The path stays cached for the next request.

Three ideas carry the whole problem:

1. **The prompts form a tree.** Shared beginnings, then branches. A radix tree stores each shared beginning once and squashes every no-branch chain into one edge.
2. **Evict leaves, never inner nodes.** An inner node is a prefix that everything below it depends on. When its last child goes, it becomes a leaf and can follow.
3. **Pin with a count.** Every running request adds 1 to each node on its path and takes it away when it finishes. Eviction skips anything above 0.

---

4. Part A: match and split (the radix tree)

Match, O(prompt length):

1. Start at the root with `i = 0`.
2. Look up the child whose edge starts with `tokens[i]` (a hash map per node, keyed by first token).
3. Compare the edge with `tokens[i..]`. If the whole edge matches, move down and continue.
4. If only part matches, **split** the edge at that point, so the matched part becomes a whole node.
5. Stop when no child starts with the next token. `i` is the reused length.

Split: an edge from `parent` to `child` holding tokens `a b c d` becomes two edges: `parent` to a new node `top` holding `a b`, and `top` to `child` holding `c d`. No tokens are added or removed. `top` copies the child's reference count, because every request pinning `child` also passes through `top`. SGLang's `_split_node` does the same (`new_node.lock_ref = child.lock_ref`).

Why split on a match, not only on insert? The matched part has to be pinned, and only a whole node can be pinned. Splitting during the walk keeps "pinned" a property of nodes.

```
private (Node Node, int Matched) WalkAndPin(Node node, int[] tokens, int i)
{
    var now = ++_clock;
    while (i < tokens.Length && node.Children.TryGetValue(tokens[i], out var child))
    {
        var common = CommonLength(child.Key, tokens, i);
        if (common < child.Key.Length) child = Split(child, common);
        node = child;
        i += common;
        node.LastAccess = now;
        node.RefCount++;
    }
    return (node, i);
}
```

---

5. Part B: pinning (reference counts)

- `Acquire` pins every node it walks through. `Dispose` on the lease walks from the last node back to the root and unpins each one. SGLang's `inc_lock_ref` / `dec_lock_ref` walk the same way.
- **Why a count:** the agent plan executor runs two candidate dossiers in parallel, and both prompts start with the same system prompt, job spec and rubric. That path has a count of 2. When the first call finishes it drops to 1 and the path is still protected.
- **Invariant:** a child is never pinned more than its parent, because every path to the child goes through the parent. The test suite's `Audit()` checks this after every scenario.
- **If everything is pinned,** nothing can be evicted, and the new tail is cached only as far as free space allows (possibly not at all). The request still runs; it just doesn't leave anything behind for the next one.

---

6. Part C: eviction (LRU over leaves, with a min-heap)

6.1 The loop (from `RadixPrefixCache.cs`)

```
var heap = new PriorityQueue<Node, long>();
foreach (var leaf in Leaves())
    if (leaf.RefCount == 0) heap.Enqueue(leaf, leaf.LastAccess);

while (freed < needed && heap.TryDequeue(out var leaf, out _))
{
    var parent = leaf.Parent!;
    parent.Children.Remove(leaf.Key[0]);
    freed += leaf.Key.Length;
    CachedTokens -= leaf.Key.Length;
    if (parent != _root && parent.Children.Count == 0 && parent.RefCount == 0)
        heap.Enqueue(parent, parent.LastAccess); // it just became a leaf
}
```

6.2 Why a heap, not the linked list from the classic LRU cache

The classic LRU cache (a hash map plus a doubly linked list) works when **every** entry can be evicted. Here only unpinned **leaves** can, and that set changes as the tree grows, splits and shrinks. Collecting the evictable leaves

and popping the oldest from a min-heap is what SGLang does too.

6.3 Invariants

1. `CachedTokens` equals the sum of all edge lengths, and never exceeds `Capacity`.
2. No empty edges. Every child is indexed under its own first token.
3. Reference counts are never negative, and a child is never pinned more than its parent.
4. A node with a reference count above 0 is never evicted.
5. After every request has finished, every count is back to 0.

6.4 Tree vs hashed blocks (vLLM)

	Radix tree (SGLang, this repo)	**Hashed blocks (vLLM)**
Lookup	walk down, comparing tokens	hash each full block with everything before it, look it up
Granularity	any number of tokens	whole blocks only
Fits a paged KV allocator	needs mapping from tokens to pages	naturally
Branching (parallel samples, agent branches)	explicit in the tree	implicit, through shared block hashes
Eviction	LRU over unpinned leaves	LRU over blocks with reference count 0

Both are correct. It mostly comes down to how the engine already manages KV memory.

6.5 Complexity

Operation	**Cost**
`Acquire`	O(L) token comparisons for a prompt of length L, plus O(edge length) to copy an edge on a split
`Insert`	O(remaining tokens), plus eviction if needed
`Evict`	O(N + (E + k) log N): N nodes visited, E unpinned leaves first put in the heap, k nodes evicted, counting the parents added when they become leaves. Worst case O(N log N). SGLang now keeps the evictable leaves in a set updated on every change, so it skips the walk (F6)
`Dispose`	O(depth)
Memory	one node per branch point, not per token

---

7. A worked run: a recruiting agent

A simulated run of a recruiting app. The prompt sizes are made up but realistic, and the numbers below are what the cache in this repo produces on them. A hiring manager asks for strong senior backend candidates and a panel pack. The plan makes five model calls. Every prompt is built from blocks, stable ones first:

Block	**Tokens**
system prompt + tools + hiring policy (SYS)	3,000
parsed job spec (JOB)	500
scoring rubric (RUB)	400

Block	Tokens
per-call tail (task line, candidate profile and so on)	50 to 2,000

Capacity is 9,000 tokens. SYS is already cached from an earlier manager.

Call	Prompt	Longest cached prefix	Reused	Computed
parse	SYS + task	SYS	3,000	50
rubric	SYS + JOB + task	SYS	3,000	600
score	SYS + JOB + RUB + candidates	SYS + JOB (split)	3,500	2,400
dossier_ana	SYS + JOB + RUB + profile	SYS + JOB + RUB (split)	3,900	1,300
dossier_bo	SYS + JOB + RUB + profile	SYS + JOB + RUB	3,900	1,100
total	**22,750**		**17,300**	**5,450**

The two dossier calls run in parallel, so the path SYS, JOB, RUB has a reference count of 2 while they run.

Later, a second manager needs 700 new tokens and only 500 are free. Eviction removes the oldest unpinned leaves: the earlier manager's parse tail (50), this run's parse tail (50), then the rubric task (100). SYS is never a candidate, because it isn't a leaf.

The test `Recruiting_agent_run_computes_5450_of_22750_prompt_tokens` checks every number above.

The same run with a timestamp at the top of the system prompt: every prompt starts differently, nothing matches, and all 22,750 tokens are computed. Prompt order matters: stable content first, the changing part last.

---

8. The tests, named after what they protect

Test	Behaviour
`Shared_beginning_is_reused_and_stored_once`	the point of the whole thing
`Splitting_an_edge_keeps_both_branches` / `A_shorter_prompt_matches_inside_an_edge`	split correctness
`Matches_the_longest_prefix_of_any_cached_prompt`	randomized check against a brute-force longest-prefix search
`Least_recently_used_goes_first` / `A_shared_prefix_outlives_its_branches`	LRU over leaves
`Evicting_a_last_child_makes_the_parent_evictable`	a parent becomes a leaf
`Too_long_to_fit_caches_as_much_as_fits`	partial insert
`A_prefix_in_use_is_never_evicted` / `When_everything_is_pinned_nothing_new_is_cached`	pinning
`A_split_keeps_the_pin_on_both_halves`	split copies the count
`All_pins_are_gone_after_every_request_finishes`	invariant 5
`Insert_after_another_request_cached_the_same_tail_adds_nothing`	no duplicate branches
`Insert_after_another_request_cached_the_same_tail_makes_it_recent`	a late `Insert` touches the tail it walks into, so LRU sees it as used
`Recruiting_agent_run_computes_5450_of_22750_prompt_tokens`	the worked run in section 7

Test	Behaviour
`Chat_traffic_skips_most_of_the_prefill`	50 users, 5 turns each: over 95% of prompt tokens reused, system prompt stored once

---

9. Going further: what real systems need next

F1. "Many requests arrive at once." Keep one owner. SGLang's scheduler is a single loop that owns the tree, the same design as the agent plan executor in this repo. If you must share it across threads, a single lock around each operation is simpler and safer than locking nodes; Dynamo's router has a concurrent radix tree for exactly this case.

F2. "Which waiting request should run next?" Prefer the one with the **longest cached prefix**. SGLang's scheduler does this to raise the hit rate. Watch for starvation of requests with no match.

F3. "We have many GPU servers." Route by prefix: keep a tree of which worker holds which prefixes and send the request where most of it is cached, balanced against load. That's NVIDIA Dynamo's KV router and llm-d's indexer.

F4. "GPU memory is too small." Add tiers. Evicted KV moves to CPU memory or SSD instead of disappearing, and comes back on a hit. Mooncake (Moonshot AI) is built around this (paper).

F5. "The KV is stored in fixed-size pages." Align edges to page boundaries, or switch to hashed blocks (section 6.4). A token-level tree over paged memory has to handle a partial last page.

F6. "Eviction is slow with a million nodes." Keep the set of evictable leaves up to date on every pin, unpin, insert and evict, instead of walking the tree each time. SGLang keeps `evictable_leaves` for this.

F7. "Cached KV for one customer must not help another." If customers share one cache, a fast response can hint that someone else sent the same prefix. Keep tenants separate: put a tenant id as the first key in the tree, so prefixes are only shared within one tenant.

F8. "Some prefixes are expensive to recompute but rarely used." Plain LRU only looks at time. A cost-aware policy weighs recompute cost (prefix length) against how recently and how often it was used. The LFU Cache problem is the frequency half of this.

---

10. Common mistakes, and why they hurt

Topic	Common mistake	What goes wrong	Better
Structure	One node per token	A 3,000-token system prompt becomes 3,000 nodes	Radix tree: one edge per no-branch run
Matching	Only splitting on insert	The matched part ends inside an edge and can't be pinned	Split during the walk
Eviction	Evict the oldest node, wherever it is	Evicting an inner node strands every branch below it	Leaves only; a parent follows when its last child goes
Eviction	Linked-list LRU over all nodes	Inner and pinned nodes sit in the list and must be skipped every time	Min-heap over unpinned leaves

Topic	Common mistake	What goes wrong	Better
Pinning	A yes/no flag	The first of two parallel requests unpins the shared path while the second still uses it	A count
Pinning	Split without copying the count	The new top node's count is too low. Releases push it below its child's count, and a request that pins only the top part can lose its protection	`top.RefCount = child.RefCount`
Insert	Insert from the root again	Another request may have cached the same tail meanwhile, giving duplicate branches	Continue from where `Acquire` stopped
Prompts	Timestamp or request id at the top	Nothing ever matches	Stable content first, changing content last

11. Run it

```
cd dsa/trees-and-tries/prefix-cache-radix-tree
dotnet test          # 26 tests: Matching (8), Eviction (5), Pinning (6), Requests (7)
```

Requires .NET 10 SDK.

Sources

- SGLang: paper (arXiv 2312.07104, NeurIPS 2024); LMSYS blog, 2024-01-17; `radix_cache.py`
- vLLM: automatic prefix caching design
- NVIDIA Dynamo: router design; llm-d: KV indexer; Mooncake: arXiv 2407.00079
- Prompt caching: Anthropic; OpenAI; Google Gemini; DeepSeek
- Outside AI: Linux `fib_trie`; Redis rax; Redis eviction; PostgreSQL buffer manager

Details come from public papers, docs and source code as of October 2026. Implementations change; read them as "publicly described as...".